

Infrastructure, Deployment, and Operability

Analyze the codebase for infrastructure and deployment assumptions.

Review:

- Environment config strategy
- Secrets/config handling
- Storage choices
- Queue infrastructure
- Database assumptions
- Containerization/deployment patterns
- Background worker execution model

Then answer:

- Is this deployable as a serious product today?
- What operational risks are visible in code/config?
- What would break first in production?
- What should be hardened before enterprise rollout?

Environment Config Strategy

What exists: Zod schema validation of all env vars at startup (config.ts). Unknown or malformed values cause `process.exit(1)` immediately. Required secrets (jwtSecret, apiKey, encryptionKey, googleClientId, googleClientSecret) are validated in production — if they contain placeholder strings or dev- prefixes, the process refuses to start.

What works well:

- Fail-fast at boot rather than at runtime is the right choice
- Single source of truth — the entire config surface is one file, one schema
- All keys are mapped explicitly from `process.env` with typed coercion

What's risky:

The LLM API keys (openaiApiKey, anthropicApiKey, groqApiKey) are not required and not validated in production. They default to `PLACEHOLDER-*` strings. The platform will boot and appear healthy with no working LLM providers. Users will hit errors at call time rather than getting a clear deployment failure. The production validation block should include at least one LLM provider being non-placeholder.

databaseUrl defaults to `postgresql://shyamshundar@localhost:5432/joallm_dev` — a developer's local username baked into the codebase default. This won't break anything (it's just a default) but it's a hygiene issue that will appear in logs.

googleClientId and googleClientSecret are required fields with no `.optional()` — the server won't boot at all without Google OAuth configured, even if you don't intend to use Google login. This is a hard dependency on a third-party service for any deployment.

Secrets / Config Handling

Encryption: AES-256-GCM with PBKDF2 key derivation (100k iterations, SHA-512). This is correct and serious cryptography. User API keys stored in `users.api_keys` (JSONB column) are encrypted at rest using this scheme. The format `salt:iv:authTag:ciphertext` is correct.

Critical gap — single encryption key, no rotation path. If `ENCRYPTION_KEY` is ever compromised or needs rotation, there is no re-encryption pathway. Every user's stored API keys would need to be decrypted with the old key and re-encrypted with the new one. No migration tooling for this exists.

JWT secret: 7-day expiry, blacklist via Redis on logout. Token blacklisting works correctly but only when Redis is available. If Redis is down, `isTokenBlacklisted()` returns false — blacklisted tokens become valid again silently. There is no fallback mechanism (e.g., short token TTL, database blacklist).

No secret rotation for JWT. A single `JWT_SECRET` means you cannot rotate it without invalidating all active sessions simultaneously. Standard practice is to support a list of valid signing secrets with a primary and legacy set.

`/api/health/config-status` leaks provider configuration state without authentication. Anyone can query this endpoint and learn which LLM providers are configured and whether mock mode is active. This is operational intelligence that should be behind admin auth.

Storage Choices

Volume storage (default): `LocalFileStorage` writes to `/app/data/uploads` on a Railway-mounted volume. The implementation is sound — directories created recursively, `ENOENT` handled on delete. The Railway `railway.toml` mounts `uploads-volume` at `/app/data`.

Critical operational risk: Railway volumes are single-replica, single-region. The volume is attached to one specific Railway container instance. When `railway.json` is set to `numReplicas: 1`, this is safe — but the moment someone scales to 2 replicas, replica 2 gets a different volume (or no volume) and file reads/writes will start returning 404s and write failures silently. There is no CDN, no shared filesystem, no object storage fallback in this configuration.

Training storage (`trainingStorageProvider`) is correctly separated — a distinct R2 bucket for RLHF signals. The bucket layout comment (`signals/{userId}/{date}.jsonl`, `feedback/{userId}/{messageId}.json`) shows deliberate design. However, the training storage falls back to the primary storage provider if R2 isn't configured, meaning training signals end up in `/app/data/uploads` on the volume alongside user documents — different concerns mixed in the same storage.

`LocalStorage.getFileUrl()` returns `/storage/${key}` — a relative URL. Nothing in the backend registers a static file serving route at `/storage/`. File download would 404 unless the frontend or nginx proxies this path. This is likely working only

because files are served through a dedicated route endpoint, not directly — but the URL format is misleading.

Queue Infrastructure

Two Redis clients, one Redis instance:

- `ioRedis` for BullMQ (`queue.ts`) — lazy connect, 3 retries, then gives up
- `redis v4` for caching (`cache.ts`) — separate connection

Both target the same `REDIS_URL`. Two connections, two client libraries, no coordination.

Graceful degradation is a false safety net. When Redis is unavailable:

- BullMQ queue is null — document uploads enqueue nothing
- The files row is written to DB (status: pending) before the enqueue attempt
- If enqueue fails, the row is orphaned permanently — no cleanup, no retry, no sync

fallback

- The upload route returns 201 to the user, who has no idea their document will never be processed

Workers co-located in the web process. `index.ts` boots `documentProcessingWorker` and `documentIndexingWorker` in the same Node.js process as the HTTP server. PDF parsing (`pdf-parse`), XLSX parsing, Word document extraction (`mammoth`), and embedding API calls all compete for the same event loop and heap. Under concurrent uploads, the HTTP server will experience elevated latency. Memory pressure from large document parsing could trigger OOM and take the entire API offline.

No dead-letter queue. BullMQ is configured with `attempts: 3`, `backoff: exponential(2s)`. After 3 failures the job goes to the BullMQ failed set. There is no monitoring, no alerting, no admin UI, no recovery path for failed jobs short of a developer

running a manual reindex script. `removeOnFail: 5` means only the last 5 failures are retained — older failures are deleted silently.

Database Assumptions

Connection pool: `max: 20 connections`, `idle_timeout: 20s`. With one replica this is appropriate. With two replicas this doubles to 40 connections to one Postgres instance — approaching typical Railway Postgres connection limits.

SSL disabled hardcoded: `ssl: false` with a comment "Railway's internal network doesn't require SSL." This is true for Railway's private networking, but any external Postgres connection (e.g., Neon, Supabase, or moving off Railway) will fail silently or require changing code. The connection string supports `?sslmode=require` via URL but the code overrides it to false.

Inspect the codebase for scale-readiness.

Evaluate:

- Horizontal scalability assumptions
- Stateful bottlenecks
- DB hotspots
- Synchronous vs asynchronous bottlenecks
- File/document processing limitations
- Model invocation bottlenecks
- Queue/retry/backpressure design

Then explain:

- What happens at 10x, 100x, and enterprise load
- Which components are most fragile
- Which refactors are mandatory for scale

Horizontal Scalability Assumptions

The entire system assumes one replica. This is explicit in `railway.json` (`numReplicas: 1`) and implicit in every stateful design decision:

- File storage writes to a Railway Volume at `/app/data/uploads` — a filesystem mount that belongs to exactly one container. A second replica gets either a separate volume (split-brain) or no mount at all.
- BullMQ workers run inside the web process. A second API replica would spawn a second pair of workers, both consuming from the same Redis queues. BullMQ handles this correctly (it's designed for multi-consumer), but the document processing workers would then be fighting with the HTTP server on two machines simultaneously — amplifying the resource contention problem rather than solving it.
- Rate limiting is in-process via `@fastify/rate-limit` with no Redis store configured. Each replica tracks its own counter independently. At 2 replicas, each allows 100 req/min — effectively doubling the global limit without intending to.
- The LLM client cache (`_userServiceCache` Map) is in-process memory. At 2 replicas, each builds its own cache independently. This is fine but worth noting: horizontal scaling here produces correct behaviour but no cross-replica cache sharing.

Stateful Bottlenecks

1. The web process holds three stateful concerns simultaneously:

- HTTP request handling (stateless, scalable)
- BullMQ document workers (stateful, blocking)
- In-memory LLM/embedding client cache (per-process state)

These three concerns have incompatible scaling requirements. HTTP wants many lightweight replicas. Workers want few heavy replicas with controlled concurrency. Mixing them means every scaling decision is a compromise.

2. Volume filesystem is the hardest statefulness. Files written by one process cannot be read by another unless they share the same mount. Railway Volumes are single-attachment. This is the load-bearing constraint that prevents horizontal scaling of the current deployment entirely.

3. Redis is correctly externalised — it is the one shared stateful dependency that works across replicas. Token blacklisting, BullMQ queues, and the cache all route through it correctly. The only problem: two separate Redis clients (ioredis + redis v4) open two connections per process, doubling connection count at scale.

DB Hotspots

document_chunks is the hottest table by far. Every RAG request scans it with a vector similarity query. Its structure:

- IVFFlat index (lists=100) created in migration 0001, overridden by HNSW in migration 0006
- Both indexes may coexist — Postgres uses one but the other still consumes write overhead on every chunk insert
- embedding column is vector(1024) — each row stores 4KB of float data
- No userId column on document_chunks — user isolation requires a JOIN through files on every query:

```
-- What every RAG search actually runs:
SELECT dc.* FROM document_chunks dc
JOIN files f ON f.id = dc.file_id
WHERE f.user_id = $userId
ORDER BY dc.embedding <=> $queryVector
LIMIT $k
```

This JOIN prevents the HNSW index from doing a pure ANN scan — Postgres must plan a filtered search, which degrades to sequential scan territory when the filter selectivity is low. At tens of thousands of chunks, the index helps. At millions of chunks across thousands of users, each user's scan touches the full index and filters post-search.

api_usage is a write hotspot. Every RAG chat, every general chat, every workflow execution appends a row. At 100 concurrent users each making 10 requests/minute, this is 1,000 inserts/minute to a table with three indexes (userId, createdAt, endpoint).

Each insert updates all three indexes. api_usage will become the slowest write in the system under load.

search_history gets double-written per RAG search request. The /search route inserts to search_history on every query hit (lines 180, 261 in rag.ts). This is synchronous and in-path — it blocks the response until the insert completes.

requireTier() is an uncached DB read on every gated request. It runs SELECT subscription_tier FROM users WHERE id = \$userId on every call. With 15 domains and many gated routes, a busy user may trigger this query 5-10 times per minute. No cache, no batching.

Synchronous vs Asynchronous Bottlenecks

RAG chat request path (fully synchronous, ~3-8 seconds):

auth middleware (Redis lookup)

→ ownership check DB query

→ getUserApiKeys DB query + decrypt

→ embed query (Cohere/OpenAI API — ~200-500ms)

→ pgvector ANN search

→ [Research: second embed + second ANN search]

→ LLM generation (1-10s depending on model + maxTokens)

→ api_usage INSERT

→ search_history INSERT (for RAG search path)

→ response

Every step is sequential. The user connection is held open for the entire duration. At 20 concurrent RAG chat requests with Research mode (2,500 max tokens, Anthropic Claude), the backend is holding 20 open HTTP connections for 10+ seconds each,

blocking 20 Postgres connections, and making 40 external API calls concurrently. Fastify can handle the connection count, but Postgres at max: 20 connections will queue the 21st request.

General chat streaming is better — it uses SSE and streams tokens as they arrive. But it still makes 39 synchronous DB operations across its path (per the grep result above).

File upload is the only properly async path — the HTTP response returns immediately after the DB insert and BullMQ enqueue. Everything after is async in the worker. This is the correct pattern and the only route that follows it.

search_history and api_usage inserts are in-path and synchronous. They add ~5-20ms each per request. At scale this is pure latency tax with no user-facing benefit. Both should be fire-and-forget (unawaited) or batched.

File/Document Processing Limitations

Worker concurrency is hardcoded and conservative:

documentProcessingWorker: concurrency: 2

documentIndexingWorker: concurrency: 1

Two documents process simultaneously. One embeds at a time (embedding is the bottleneck — it's an external API call). At 10 concurrent users each uploading a document, 8 jobs sit in queue waiting. Users see "processing" status with no ETA.

Buffer serialisation through Redis is fragile. The processing worker job payload includes the raw file buffer:

```
// queue.ts worker deserialisation
const fileBuffer = Buffer.isBuffer(buffer) ? buffer : Buffer.from((buffer as any).data || buffer);
```

Redis serialises Buffer as a plain object. The worker reconstructs it with a defensive cast. A 20MB PDF becomes a ~26MB JSON payload sitting in Redis. Redis is not designed as a binary object store. At 10 concurrent 20MB uploads, the queue payload alone is 260MB in Redis memory. The default Railway Redis plan is 25MB.

No file size validation before queueing. The upload route accepts up to 50MB (multipart limit). A 50MB file becomes a ~65MB Redis payload. One such upload could exhaust the Redis memory limit entirely.

Embedding is per-chunk, sequential within a job. A 200-page PDF produces ~400 chunks. Each chunk triggers one Cohere API call. That's 400 sequential API calls per document. At Cohere's free tier (100 calls/minute), one large document takes 4+ minutes.

No batching API is used despite Cohere's embed endpoint supporting arrays of up to 96 texts per call.

Model Invocation Bottlenecks

No request queuing or concurrency cap for LLM calls. Every RAG chat request triggers an LLM API call directly. 50 concurrent Research-mode requests = 50 simultaneous LLM calls. External providers throttle at the account level. When the rate limit hits, calls fail with a 429 from the provider — this surfaces as a 500 to the user with no retry.

No streaming in RAG chat. /api/rag/chat waits for the full completion before sending any bytes. A Research-mode response with maxTokens=2500 on Claude holds the HTTP connection for 10-20 seconds. Fastify's default request timeout will close the connection before the LLM finishes for very long responses.

No token budget enforcement. Research mode sends up to 25 chunks to the LLM without checking whether the assembled context fits the model's context window. A context overflow either: (a) silently truncates in the API call, degrading quality, or (b) throws a 400 from the provider that the route doesn't handle gracefully.

EmbeddingService is not cached — unlike LLMService which now has the client cache, EmbeddingService still creates new Cohere and OpenAI clients per call. This means 400 Cohere API calls for one large document each spin up a new HTTP client.

Queue / Retry / Backpressure Design

BullMQ config as deployed:

attempts: 3

backoff: exponential, 2s initial

removeOnComplete: 10 (keeps last 10 successful jobs)

removeOnFail: 5 (keeps last 5 failed jobs)

No backpressure. The queue accepts any number of jobs. A user who uploads 50 documents simultaneously gets 50 jobs enqueued immediately. The 2-concurrent processing worker will drain them over ~25 processing cycles, but nothing prevents the queue from growing unbounded. No maxSize on the queue, no rate limit on upload count per user per time window.

No dead-letter queue monitoring. Failed jobs after 3 attempts sit in BullMQ's failed set. removeOnFail: 5 means only 5 failed jobs are retained. If 20 documents fail to process, 15 failure records are silently deleted. There is no alerting, no admin recovery workflow.

Job payload in Redis (not a job reference). The correct pattern for large payloads is to store the file in object storage first, then enqueue a reference (fileId). The current pattern puts the buffer directly in the job. This ties Redis memory to upload volume — a queue infrastructure concern is conflated with storage.

No job timeout configured. A document that hangs (e.g., a corrupt PDF causing pdf-parse to spin) will hold a BullMQ slot indefinitely. The worker has concurrency: 2 — one hung job consumes 50% of processing capacity forever until the process restarts.

What Happens at 10x, 100x, and Enterprise Load

10x Current Load (~100 active users, ~1,000 req/hour)

- Postgres connection pool (max: 20) starts queuing requests. requireTier() uncached reads and api_usage inserts compete for the same 20 connections. P99 latency on gated routes rises noticeably.
- Rate limiting (100 req/min global, in-memory) becomes inaccurate — a power user can saturate the limit before others get a share, but the global counter is shared so normal users get incidentally throttled.
- Document processing queue begins backing up during peak upload windows. Users waiting on "processing" status with no feedback.
- Redis memory pressure from buffer-in-payload starts appearing if users upload large files concurrently.
- Everything still works. Degraded but functional.

100x Current Load (~1,000 active users, ~10,000 req/hour)

- Postgres connection pool exhausted. With 20 max connections and multiple uncached reads per request (requireTier, getUserApiKeys, ownership checks), requests start timing out at the DB layer. The 10-second connect_timeout begins triggering. Users see 500s on authenticated routes.
- Workers OOM the web process. 2 concurrent processing workers + HTTP traffic at this volume will breach the 2GB heap limit (NODE_OPTIONS=--max-old-space-size=2048). Node.js OOM-kills the process. Railway restarts it. All in-flight jobs are dropped (BullMQ re-queues them on reconnect, but the restart takes the API down for 30-60 seconds).
- LLM provider rate limits hit. 1,000 users each doing 10 RAG chats/hour = ~167 concurrent LLM calls/minute. Groq's free tier is 30 RPM. Provider 429s cascade into user-facing errors.
- api_usage table write contention. 10,000 inserts/hour with 3 indexes. This table becomes the slowest Postgres object. Long-running vector searches block connections; api_usage inserts pile up behind them.
- The platform is partially functional but unreliable. Most users experience intermittent errors.

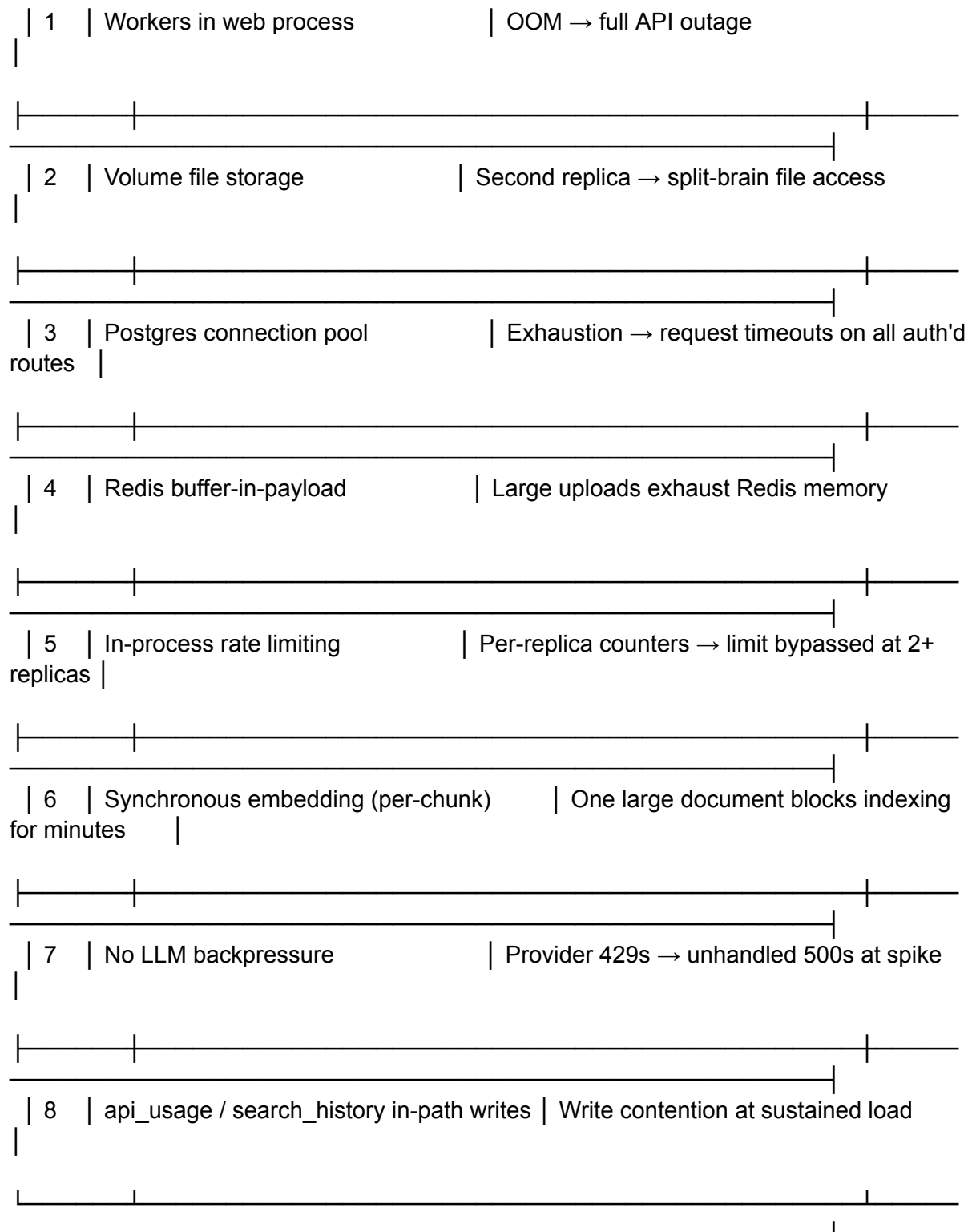
Enterprise Load (~10,000+ users, sustained)

- Volume storage is completely incompatible. Cannot run more than 1 replica. The only way to scale the backend is vertically, which hits Railway's single-instance limits quickly.
- pgvector IVFFlat/HNSW at tens of millions of chunks. With lists=100, IVFFlat needs to be rebuilt when the dataset grows significantly (rule of thumb: lists = sqrt(num_rows)). The HNSW index m=16, ef_construction=64 (migration 0006 defaults) provides good recall but index build time for millions of rows is hours. Index maintenance on every chunk insert is significant overhead.
- Single Postgres instance. No read replicas. All vector searches, all chat reads, all file metadata queries hit one primary. pgvector searches are CPU-intensive — they compete directly with OLTP workloads on the same machine.
- No tenant isolation in data model. Every query includes WHERE user_id = ? but there is no organisation-level partitioning. A single misconfigured query (missing WHERE clause) could return another tenant's data. Partition-by-user or row-level security becomes mandatory.
- The system cannot reach enterprise load in its current form. It would need to be substantially re-architected before approaching this range.

Most Fragile Components

Ranked by likelihood of being the first failure at scale:

Rank	Component	Failure Mode



Mandatory Refactors for Scale

Must happen before scaling beyond 1 replica:

1. Extract workers to a separate Railway service.

The web API and document processing must be split. The worker service runs `documentProcessingWorker` + `documentIndexingWorker` only, with no HTTP server. Concurrency can then be tuned independently. This is a single `index.ts` change + a second Railway service definition.

2. Store file buffer in object storage before enqueueing; enqueue only the `fileId`.

The upload route should write the buffer to R2/S3, then enqueue `{ fileId }`. The worker reads from storage. This removes the Redis memory dependency on file size entirely, and enables the file to survive a Redis flush.

3. Move to R2 or S3 as the default storage provider.

Volume storage must not be the production default. Object storage is the prerequisite for horizontal scaling.

4. Redis-backed rate limiting.

Replace `@fastify/rate-limit` with its Redis store option (`@fastify/rate-limit` supports this natively). This makes rate limits accurate across replicas at zero code change cost.

Must happen before sustained multi-user load:

5. Cache `requireTier()` and `getUserApiKeys()` in Redis.

5-minute TTL keyed by `userId`. These two calls alone account for 2 DB round-trips per authenticated request. At 1,000 users this is the primary driver of connection pool exhaustion.

6. Make `api_usage` and `search_history` writes fire-and-forget.

Remove `await` from both. Wrap in a `setImmediate` or use `Promise.allSettled` without awaiting. These writes are observability data — they must not be in the response critical path.

7. Batch Cohere embeddings during document indexing.

Replace the per-chunk sequential `embeddingService.generateEmbedding()` call with batched calls (up to 96 texts per Cohere request). A 400-chunk document goes from 400 API calls to 5. This alone reduces indexing time by ~95% and eliminates most of the Cohere rate-limit exposure.

8. Add a `userId` column directly to `document_chunks`.

Eliminates the JOIN in every vector search. Allows the HNSW index to work on a pre-filtered subset. This is a migration + a one-time backfill from the `files` table.

Must happen before enterprise contracts:

9. Add per-job timeout to BullMQ.

`timeout: 300000` (5 minutes) on processing jobs, `timeout: 120000` on indexing. Stale jobs fail cleanly and release their worker slot.

10. Implement a dead-letter queue with alerting.

removeOnFail: 50 and a monitoring route that exposes failed job count. Any failed job count > 0 should trigger an alert. Recovery should be possible without developer intervention (a /api/admin/reindex/:fileId endpoint already exists via the scripts — surface it properly).

11. Add statement_timeout to Postgres connection.

Set statement_timeout = 30000 (30 seconds) on the connection string or per-session for vector queries. Prevents runaway scans from holding connections indefinitely.